

Sesiunea 1.2: Cele 7 Principii ale Testării (Studiu Individual)

Partea 1: Cele 7 Principii ale Testării (Standardul ISTQB).....	1
1. Testarea demonstrează prezența defectelor, nu absența lor	2
2. Testarea exhaustivă este imposibilă	2
3. Testarea timpurie economisește timp și bani (Early Testing)	2
4. Gruparea defectelor (Defect Clustering).....	2
5. Paradoxul Pesticidului	2
6. Testarea depinde de context	3
7. Eroarea / Iluzia absenței erorilor	3
Partea 2: QA (Quality Assurance) vs. QC (Quality Control)	3
Asigurarea Calității (QA - Quality Assurance)	3
Controlul Calității (QC - Quality Control)	3
Partea 3: Nivelurile de Independență în Testare	4
Avantajele și Dezavantajele Independenței	4

 Acest modul de studiu individual este esențial atât pentru formarea mindset-ului corect de QA, cât și ca bază teoretică fundamentală pentru certificarea **ISTQB Foundation Level**. Aici explorăm regulile de aur ale testării, delimităm clar rolurile din industrie și înțelegem de ce obiectivitatea este cea mai de preț calitate a unui tester.

Partea 1: Cele 7 Principii ale Testării (Standardul ISTQB)

Aceste principii nu sunt doar teorie; ele sunt „avertismente” scrise cu sânge (și bugete depășite) de generațiile anterioare de ingineri software. Ele ne temperează așteptările și ne ghidează strategia.

1. Testarea demonstrează prezența defectelor, nu absența lor

- **Concept:** Testarea poate dovedi clar că un soft *are* bug-uri (când unul pică). Însă, oricât de mult ai testa, nu poți dovedi matematic că softul *nu are niciun bug*.
- **Semnificație:** Testarea reduce probabilitatea ca defectele să rămână ascunse, dar chiar și după luni de testare, un sistem nu poate fi declarat „100% bug-free”.

2. Testarea exhaustivă este imposibilă

- **Concept:** Să testezi absolut toate combinațiile de date de intrare, precondiții și rute prin aplicație este imposibil (cu excepția unor cazuri triviale).
- **Soluția QualiAdept:** În loc să încercăm imposibilul, folosim **analiza riscului și prioritizarea**. Testăm mai mult acolo unde impactul unui eșec este mai mare.

3. Testarea timpurie economisește timp și bani (Early Testing)

- **Concept:** Activitățile de testare ar trebui să înceapă cât mai devreme în ciclul de viață al dezvoltării software (încă din faza de cerințe - *Static Testing*).
- **Motivul:** Un defect logic găsit pe hârtie costă 1 Euro să fie reparat. Același defect, găsit după ce codul a fost scris, testat și lansat în producție, poate costa 10.000 Euro și daune de imagine. Această practică mai poartă numele de „**Shift Left**”.

4. Gruparea defectelor (Defect Clustering)

- **Concept:** Un număr mic de module conține, de obicei, majoritatea defectelor descoperite într-un sistem.
- **Legea lui Pareto (Regula 80/20):** Aproximativ 80% din bug-uri se găsesc în 20% din cod. Dacă găsești un bug într-o anumită secțiune a aplicației, fii foarte atent acolo, pentru că e foarte probabil să mai găsești și altele (bug-urilor le place să „stea în haită” pentru că de obicei o zonă cu probleme a fost scrisă de un programator neexperimentat sau are o logică foarte complicată).

5. Paradoxul Pesticidului

- **Concept:** Dacă rulezi exact aceleași teste mereu și mereu, la un moment dat ele nu vor mai

găsi defecte noi. Sistemul devine „imun” la ele.

- **Soluția:** Cazurile de test trebuie revizuite și actualizate periodic. Trebuie să scriem teste noi pentru a acoperi părți noi din software, la fel cum fermierii trebuie să schimbe pesticidele pentru că insectele devin rezistente.

6. Testarea depinde de context

- **Concept:** Nu există o „rețetă universală” de testare.
- **Exemplu:** O aplicație bancară (unde siguranța financiară e critică) se testează complet diferit, cu alte rigori și reglementări, comparativ cu un joc pe mobil sau un site de prezentare a unei florării.

7. Eroarea / Iluzia absenței erorilor

- **Concept:** Degeaba găsim și reparăm 10.000 de defecte dacă sistemul pe care l-am construit este inutilizabil, nu răspunde nevoilor de business ale clientului sau este inferior concurenței.
- **Concluzie:** Calitatea nu înseamnă doar lipsa bug-urilor tehnice, ci și satisfacerea nevoii reale a utilizatorului.

Partea 2: QA (Quality Assurance) vs. QC (Quality Control)

Foarte des, termenii sunt folosiți interschimbabil pe piața muncii (ex: te angajezi ca „QA Tester”), însă tehnic, ei reprezintă lucruri distincte.

Asigurarea Calității (QA - Quality Assurance)

- **Orientare:** Pe Proces.
- **Obiectiv:** Prevenirea introducerii defectelor.
- **Activități:** Stabilirea proceselor, a standardelor de codare, alegerea tool-urilor (ex: Jira, TestRail), definirea metodologiei (Agile, Scrum). QA-ul se asigură că modul în care lucrăm este corect.
- *Este o activitate proactivă.*

Controlul Calității (QC - Quality Control)

- **Orientare:** Pe Produs.
- **Obiectiv:** Găsirea defectelor în produsul deja construit, înainte ca acesta să ajungă la client.

- **Activități:** Execuția efectivă a testelor, raportarea bug-urilor, validarea funcționalităților.
- *Este o activitate reactivă.*

💡 Analogia QualiAdept: „Fabrica de prăjituri”

- **QA (Asigurarea Calității)** este elaborarea rețetei perfecte, asigurarea că bucătării poartă mănuși, că se folosesc cele mai bune ingrediente și că temperatura cuptorului este setată corect. Ne asigurăm că *procesul* va produce o prăjitură bună.
- **QC (Controlul Calității)** înseamnă ca, după ce prăjitura a ieșit din cuptor, să o guști și să verifici dacă e arsă, dacă e destul de dulce și dacă arată bine, înainte să o dai clientului. Aici are loc testarea propriu-zisă.

Partea 3: Nivelurile de Independență în Testare

De ce nu este bine ca un programator să-și testeze singur codul? Răspunsul stă în psihologie: **Confirmation Bias** (biasul de confirmare). Oamenii sunt programați să caute dovezi care să le susțină convingerile. Un dezvoltator va rula codul pe „Happy Path” pentru a demonstra că *funcționează*, în timp ce un tester independent va încerca rute neobișnuite pentru a demonstra că *se poate strica*.

Conform ISTQB, independența este structurată pe mai multe niveluri (de la cea mai slabă la cea mai puternică):

- **Nivelul 0 (Fără independență):** Autorul codului (Dezvoltatorul) își testează propriul cod. *Eficiență foarte scăzută în găsirea erorilor logice.*
- **Nivelul 1:** Un alt dezvoltator din aceeași echipă testează codul (Peer Review). *Eficiență medie, dar ambii au același mindset tehnic.*
- **Nivelul 2:** O echipă de testare (QA) independentă, dar care face parte din aceeași companie și din același proiect. *Acesta este standardul industriei (rolul pe care îl veți ocupa).*
- **Nivelul 3 (Independență Maximă):** Testeri dintr-o organizație externă (firme de consultanță, auditori). Ei nu sunt influențați de presiunile manageriale interne sau de afinitățile față de colegi.

Avantajele și Dezavantajele Independenței

- **Avantaje:** Testerii independenți observă defecte și defectuoziități în documentație pe care creatorii le ignoră din obișnuință sau din cauza implicării emoționale.
- **Dezavantaje (ale independenței extreme):** Testerii externi sau prea izolați pot avea un deficit de cunoștințe despre business-ul specific al aplicației, iar comunicarea cu echipa de dezvoltare poate deveni dificilă (fenomenul „noi vs. ei”).

*Material conceput pentru studiul individual **QualiAdept**. Asigurați-vă că stăpâniți cele 7 principii, ele reprezentând un procentaj semnificativ din întrebările examenului de certificare ISTQB.*